Dual Topology Views

SolariNet · a physical (link-level) map and a monitoring (vantage→target) map, both carrying live node & interconnect health · 2026-07-04

> **Headline.** The spec calls for a *dual* topology view — one showing the physical L2 interconnects (gateway → switch/AP → host), the other showing the monitoring system and its buckets (server → monitor vantages → probe targets). Both existed as backend projections but the dashboard rendered a *hard-coded fixture* that never called `/api/topology` — the same read-layer gap that left the graphs blank. Both views are now **live and data-driven**, laid out by a ported network-map graph engine, and every node *and* every interconnect is colored and labelled by real health.

What was wrong

SYMPTOM	ROOT CAUSE	FIX
Topology map showed a static demo regardless of real fleet	The `Topology` component rendered `data.jsx` fixtures with positions hard-coded to fixture gear ids (`sw-core` , `gw`); it never fetched `/api/topology`	Component fetches the live endpoint for both views; offline mode synthesizes the same contract shape from fixtures (no hard-coded ids)
No health on the wires	Edges carried only a static kind; no probe outcome, RTT/loss, link operStatus or utilization	Both projections decorate every edge with a derived `health` + human `label`
Layout couldn't generalize	Positions were bespoke per fixture; arbitrary live topology had nowhere to go	A ported MST + radial + force-directed engine positions any graph deterministically
Gear→gear uplinks were sparse	The LLDP-over-SNMP walk was a stub (returned zero neighbours)	Real `lldpRemTable` walk by numeric OID feeds the existing (tested) normalizer

Network view returned 6,026 edges / 1.6 MB	`lldpEdge` is append-only; the projection emitted one edge per historical observation	Windowed query keeps only the latest row per `(nodeId, gearId, localIf)` — 6,027 rows → 3 links

The two views

Network — physical interconnects

`GET /api/topology?view=network`. Nodes are **network gear** (gateway/switch/AP/router) and **hosts**; edges are gear→gear `uplink` chains and host→gear `attaches` links from `lldpEdge` (with `node.uplinkGearId` as fallback). Each gear carries an interface rollup (`ifUp/ifDown/ifTotal`) from `gearInterfaceCurrent` ; each interconnect is matched to its interface (join on `gearId` and `ifName ∈ {peerPort, localIf}`) for `operStatus` , in/out rate, and utilization.

Monitoring — the monitoring system and its buckets

`GET /api/topology?view=monitoring`. Nodes are the fleet (server/monitor/client) plus **probe-target** pseudo-nodes; edges are `monitorNode → target` assignments from `probeCurrent` , decorated with the live `outcome` , RTT, jitter and loss.

Health vocabulary

Both views share one classification, driving both fill (nodes) and stroke (edges):

HEALTH	NODE / GEAR	INTERCONNECT
up	state `up` ; all interfaces up; seen < 180s ago	probe `ok` ; or operStatus 1 & util < 85%
degraded	mixed interface up/down; or stale > 180s	refused / tls_fail / proto_err; loss ≥ 10%; or util ≥ 85%
down	state `down` ; all interfaces down; stale > 900s	timeout / unreachable / dns_fail; or operStatus 2

unknown	never seen / no state	no probe or interface data to score the link

Staleness is computed in SQL with `TIMESTAMPDIFF(SECOND, lastSeenAt, NOW())` so there is no PHP/DB timezone drift. Interconnect utilization is `round(max(inRateKbps, outRateKbps) / (speedMbps·1000) · 100)`.

Layout engine — reusing network-map

We assessed the network-map project for reuse. Verdict: **reuse the layout algorithm and its visual language; do not import the scanner or the 3D renderer.**

NETWORK-MAP PIECE	DECISION	WHY
Kruskal MST + BFS radial + Fruchterman–Reingold force refine (`layout.c` , `graph.c`)	Port to JS	Exactly the data-driven positioning the dashboard needed; generalizes to any live topology
Host classification & connection-medium colors	Adopt language	Informs the node glyphs and wired/wireless styling
C scanner (nmap / ARP / UniFi / mDNS / ICMP)	Do not import	SolariNet already discovers via agent self-report + SNMP; a parallel scanner would be shoehorning
3D three.js / WebVR HTML renderer	Do not import	Wrong fit for the read-only React SPA; heavyweight for the primary operator view

The port lives in `dashboard/public/layout.jsx` as `window.SolariTopoLayout.compute(nodes, edges, opts)` — pure, deterministic (no `Math.random`), and `require()` -able for headless tests. MST edge weights prefer L2 tree edges (`uplink` / `attaches` = 1) over monitoring edges (2); the root is the gateway gear (network) or the server node (monitoring); BFS gives depth, radial placement seeds positions, and ~120 force-refine iterations even out spacing with the root pinned.

LLDP-over-SNMP topology discovery

The deep managed-gear walk is now implemented (behind `SOLARI_WITH_SNMP`, numeric OIDs, no libnetsnmp link — mirroring the IF-MIB poller). It walks `lldpRemTable` (`.1.0.8802.1.1.2.1.4.1.1.*`) via `snmpbulkwalk -v2c -Onq`, groups columns by the `(localPortNum, remIndex)` index tail, decodes chassis/port ids by subtype (MAC → colonated hex), converts the `lldpRemSysCapEnabled` BITS string to the internal capability bitmap (bridge → switch, WLAN-AP → ap, …), and resolves local interface names from `lldpLocPortId` . Each neighbour flows through the already-unit-tested `topoNormalizeMibRecord` into `networkGear` + `lldpEdge` . Driver: `solariSnmpPoll --lldp` reads every gear's `mgmtIp` (and per-gear `snmpCommunity`) and deep-walks each.

> **Environment note.** Populated gear→gear uplinks require managed devices that answer SNMP with the LLDP-MIB enabled (e.g. UniFi with SNMP + LLDP toggled on). Until then the network view is built from agent self-reported uplinks; the SNMP path enriches it in place via the same upsert key.

Surfaces touched

LAYER	CHANGE
Backend (C)	`serverTopology.c` real LLDP-MIB walk + `serverTopologyDeepWalkAll` ; `snmpPollMain.c` `--lldp` ; new unit suite `test_server_topo_snmp` (9 tests)
API (PHP, read-only)	`routes/topology.php` — both projections decorate nodes+edges with health/labels; interface joins; append-only `lldpEdge` collapsed to latest-per-link
Dashboard (JSX)	New `layout.jsx` engine; `screens2.jsx` `Topology` rewritten live + offline-synth; `index.html` loads the engine; health CSS
Tests	C: 25/25 (SNMP) & server-no-SNMP 25/25. JS: `tests/dashboard/test_layout.js` (determinism + finite positions). All dashboard JSX transpiles clean.

Verification

- SNMP server build — 25/25 ctest; default no-SNMP build — clean, seam a no-op returning `ERR_PLATFORM`.
- Layout engine — byte-identical positions across repeated calls; all coordinates finite; disconnected nodes ring-placed.
- Live API against production DB — monitoring: 20 nodes / 36 edges; network: 4 nodes / **3** edges (post-fix), every node and edge carrying `health` + `label`.

SolariNet · `feat/topology-dual-view` · standard C backend + PHP/JSX dashboard · 2026-07-04